Dynamic Programming Study Guide

1. Quick Overview of Recursion

What is Recursion?

Recursion is a programming technique where a function calls itself to solve a problem by breaking it down into smaller, similar subproblems.

Key Components of Recursion

- **Base Case**: The condition that stops the recursion
- **Recursive Case**: The function calling itself with modified parameters

Example: Factorial Function

```java
public static int factorial(int n) {
    // Base case
    if (n <= 1) {
        return 1;
    }
    // Recursive case
    return n * factorial(n - 1);
}
```

Example: Fibonacci Sequence

```java
public static int fibonacci(int n) {
    // Base cases
    if (n <= 1) {
        return n;
    }
    // Recursive case
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Problems with Simple Recursion

- **Overlapping Subproblems**: Same calculations repeated multiple times

- **Exponential Time Complexity**: Can lead to very slow execution
- **Stack Overflow**: Too many recursive calls can exhaust memory

2. Introduction to Dynamic Programming

What is Dynamic Programming?

Dynamic Programming (DP) is an optimization technique that solves complex problems by breaking them down into simpler subproblems and storing the results to avoid redundant calculations.

Core Principles

- **Optimal Substructure**: The optimal solution contains optimal solutions to subproblems
- **Overlapping Subproblems**: The same subproblems are solved multiple times in naive approaches

Two Main Approaches

Top-Down Approach (Memoization)

Store results of expensive function calls and return cached results when same inputs occur again.

```java
import java.util.HashMap;
import java.util.Map;

public class FibonacciMemoization {
    private static Map<Integer, Integer> memo = new HashMap<>();

    public static int fibonacci(int n) {
        if (n <= 1) {
            return n;
        }

        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        int result = fibonacci(n - 1) + fibonacci(n - 2);
        memo.put(n, result);
        return result;
    }
}
```

Bottom-Up Approach (Tabulation)

Build solutions iteratively from smallest subproblems to the main problem.

```java
public class FibonacciTabulation {
    public static int fibonacci(int n) {
        if (n <= 1) {
            return n;
        }

        int[] dp = new int[n + 1];
        dp[0] = 0;
        dp[1] = 1;

        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }
}
```

3. Importance of Dynamic Programming

Performance Benefits

- **Time Complexity Reduction**: Converts exponential time algorithms to polynomial time
- **Space-Time Tradeoff**: Uses additional memory to achieve significant time savings

Comparison: Fibonacci Implementation

Approach	Time Complexity	Space Complexity
Simple Recursion	O(2^n)	O(n)
Memoization	O(n)	O(n)
Tabulation	O(n)	O(n)

Real-World Applications

- **Optimization Problems**: Finding minimum cost, maximum profit
- **Counting Problems**: Number of ways to achieve a goal

- **Decision Problems**: Whether a solution exists
- **Algorithm Design**: Shortest path, longest common subsequence

Example: Climbing Stairs Problem

```java
// Problem: How many ways to climb n stairs if you can take 1 or 2 steps?
public class ClimbingStairs {
    // DP Solution
    public static int climbStairs(int n) {
        if (n <= 2) {
            return n;
        }

        int[] dp = new int[n + 1];
        dp[1] = 1;
        dp[2] = 2;

        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }
}
```

4. Identification of Dynamic Programming

When to Use Dynamic Programming?

A problem is suitable for DP if it satisfies these conditions:

Optimal Substructure

The optimal solution to the problem contains optimal solutions to subproblems.

Example: In shortest path problems, if the shortest path from A to C goes through B, then the path from A to B and B to C must also be shortest paths.

Overlapping Subproblems

The recursive solution solves the same subproblems multiple times.

Test: Draw the recursion tree - if you see repeated nodes, there are overlapping subproblems.

Common DP Problem Patterns

1. Linear DP

Problems that can be solved by making decisions at each step.

```java
// Example: Maximum sum of non-adjacent elements
public static int maxSum(int[] arr) {
    int n = arr.length;
    if (n == 0) return 0;
    if (n == 1) return arr[0];

    int[] dp = new int[n];
    dp[0] = arr[0];
    dp[1] = Math.max(arr[0], arr[1]);

    for (int i = 2; i < n; i++) {
        dp[i] = Math.max(dp[i-1], dp[i-2] + arr[i]);
    }

    return dp[n-1];
}
```

2. Grid DP

Problems involving 2D grids or matrices.

```java
// Example: Minimum path sum in grid
public static int minPathSum(int[][] grid) {
    int m = grid.length;
    int n = grid[0].length;
    int[][] dp = new int[m][n];

    dp[0][0] = grid[0][0];

    // Fill first row
    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j-1] + grid[0][j];
    }

    // Fill first column
    for (int i = 1; i < m; i++) {
        dp[i][0] = dp[i-1][0] + grid[i][0];
    }

    // Fill rest of the table
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = Math.min(dp[i-1][j], dp[i][j-1]) + grid[i][j];
        }
    }

    return dp[m-1][n-1];
}
```

Steps to Solve DP Problems

1. **Identify the problem structure**: Look for optimal substructure and overlapping subproblems
2. **Define the state**: What parameters uniquely identify a subproblem?
3. **Write the recurrence relation**: How does the solution relate to smaller subproblems?
4. **Determine base cases**: What are the simplest cases?
5. **Choose implementation approach**: Top-down (memoization) or bottom-up (tabulation)
6. **Optimize space if possible**: Can you reduce space complexity?

Quiz Questions

1. What is the main advantage of using Dynamic Programming over simple recursion? a) It uses less memory b) It avoids redundant calculations by storing results c) It is easier to implement d) It works for all recursive problems

Answer: b) It avoids redundant calculations by storing results

2. Which of the following is NOT a requirement for a problem to be suitable for Dynamic Programming? a) Optimal substructure b) Overlapping subproblems c) Greedy choice property d) Recursive nature

Answer: c) Greedy choice property

3. In the top-down approach of Dynamic Programming, what technique is primarily used? a) Iteration b) Memoization c) Tabulation d) Recursion elimination

Answer: b) Memoization

4. What is the time complexity of the naive recursive Fibonacci implementation? a) O(n) b) $O(n^2)$ c) O(2^n) d) O(log n)

Answer: c) O(2^n)

5. Which approach builds the solution from the smallest subproblems up to the main problem? a) Top-down approach b) Memoization c) Bottom-up approach d) Divide and conquer

Answer: c) Bottom-up approach

Practice Coding Questions

1. Coin Change Problem Write a Java method that finds the minimum number of coins needed to make a given amount. You have coins of denominations [1, 5, 10, 25] and an unlimited supply of each.

```java
public static int coinChange(int amount) {
    // Your implementation here
}
```

2. Longest Common Subsequence Given two strings, find the length of their longest common subsequence using dynamic programming.

```java
public static int longestCommonSubsequence(String text1, String text2) {
    // Your implementation here
}
```

3. House Robber You are a robber planning to rob houses along a street. Each house has a certain amount of money. You cannot rob two adjacent houses. Find the maximum amount you can rob.

```java
public static int rob(int[] nums) {
    // Your implementation here
}
```

4. Unique Paths A robot is located at the top-left corner of an m x n grid. The robot can only move either down or right. How many possible unique paths are there to reach the bottom-right corner?

```java
public static int uniquePaths(int m, int n) {
    // Your implementation here
}
```

5. Maximum Subarray Sum Find the contiguous subarray within a one-dimensional array of numbers that has the largest sum using dynamic programming approach.

```java
public static int maxSubArray(int[] nums) {
    // Your implementation here
}
```